

Software Project Management

The Software Project Management (SPM) process spans five fundamental management process groups:

1. Initiating Phase (Feasibility & Charter)

The Initiating phase focuses on evaluating whether a software project is worth undertaking before committing financial capital and human resources.

- Strategic & Technical Assessment:** Verifies that the project supports long-term business goals and competitive strategy, while ensuring the organization possesses or can acquire the necessary technology, infrastructure, and technical skills.
- Cost-Benefit Analysis (CBA) & Cash Flow Forecasting:** Compares expected development and operational costs against anticipated financial benefits using financial metrics like Payback Period, Return on Investment (ROI), and Net Present Value (NPV).
- Initial Risk Assessment:** Identifies high-level uncertainties regarding technology, market, and resources.
- Project Charter:** Formally authorizes the project, establishes its core objectives, and defines overarching constraints.

2. Planning Phase (WBS, Schedule, Budget & Strategy)

The Planning phase converts broad project objectives into detailed, actionable operational blueprints.

- Work Breakdown Structure (WBS):** Decomposes the total project scope into manageable deliverables and work packages using product-oriented, process-oriented, or hybrid styles while obeying the 100% rule.
- Software Effort & Cost Estimation:** Predicts required effort (in Person-Months) and duration using structural methods (Expert Judgment, Analogy, Top-Down, Bottom-Up) or parametric models (Function Point Analysis, COCOMO).
- Activity Sequencing & Scheduling:** Defines logical task dependencies (FS, SS, FF, SF), constructs network diagrams (PDM/AON or ADM/AOA), calculates activity floats, and identifies the **Critical Path**.
- Scheduling Methods & Schedule Compression:** Employs deterministic (CPM) or probabilistic 3-time-estimate methods (PERT). If deadlines are tight, schedule compression techniques like **Crashing** (adding resources) or **Fast Tracking** (parallelizing sequential tasks) are applied.
- Process Model & Technology Selection:** Chooses an appropriate lifecycle model (Waterfall, Agile, or Spiral/Risk-Driven) and tech stack.
- Risk, Quality & SCM Planning:** Establishes the Risk Register and threat response strategies (Avoidance, Mitigation, Transfer, Acceptance), sets Quality Assurance (SQA) standards, and establishes Software Configuration Management (SCM) baselines.

3. Executing Phase (Development & Implementation)

In the Executing phase, the software development team carries out the planned activities to build, test, and integrate the logical components.

- Software Synthesis & Coding:** Developers write, document, and integrate code modules according to architectural requirements.
- Verification & Quality Assurance:** Conducts peer code reviews, inspections, unit testing, and integration testing to detect defects early.
- SCM Check-out & Modification:** Developers check out configuration items (CIs) from the repository, implement approved changes, perform developer testing, and check items back in.
- Leadership & Team Direction:** Allocates responsibilities according to capabilities, handles interpersonal conflicts, and maintains team motivation and coordination.

4. Monitoring & Controlling Phase (Tracking & Variance Management)

Running in parallel with execution, this phase tracks project velocity, measures deviations from baselines, and applies corrective actions.

- Schedule & Progress Control:** Tracks activity completion, milestone status, and critical path dependencies.
- Configuration Change Control:** Processes formal Change Requests (CRs). The **Configuration Control Board (CCB)** conducts impact analysis on cost, schedule, and quality before approving or rejecting scope modifications.
- Risk Monitoring:** Tracks risk triggers, monitors changing probabilities/impacts, and executes contingency plans.
- Quality & Performance Metrics:** Evaluates project health using core metrics like Walker Royce's 7 metrics (Work & Progress, Budgeted Cost, Staffing Dynamics, Change Traffic, Breakage, Rework, and MTBF/Maturity).

5. Closing Phase (Handover & Postmortem)

The Closing phase brings the software project to a formal and orderly conclusion.

- Client Handover & Acceptance:** Conducts acceptance testing alongside **Functional Configuration Audits (FCA)** and **Physical Configuration Audits (PCA)** to verify that the final deliverable matches all approved requirements and documentation.
- Administrative Closure:** Finalizes financial accounts, closes vendor contracts, and releases project staff and resources.
- Postmortem Evaluation & Lessons Learned:** Documents project performance, estimation accuracy, and team feedback in historical archives to improve future project planning.

The 9 Key Risk Identification Techniques

- Brainstorming:** Project team members and stakeholders collaborate to discuss and surface potential risks based on their collective knowledge and experience.
- Checklists:** Referencing standard lists of risks encountered in previous projects to check if similar threats apply to the current project.
- SWOT Analysis:** Examining project Strengths, Weaknesses, Opportunities, and Threats to uncover latent risks and opportunities.
- Historical Information:** Reviewing documentation, logs, and postmortem records from past projects to identify recurring problems.
- Expert Judgment:** Consulting subject-matter experts who leverage specialized domain experience to spot non-obvious risks.
- Delphi Technique:** Requesting independent, anonymous risk identification and evaluation from a panel of experts over multiple iterative rounds to reach consensus without personal bias.
- Scenario Analysis:** Considering hypothetical future events—such as unexpected key staff resignations, server failures, or major requirement changes—to evaluate potential impacts.
- Cause-and-Effect Analysis:** Tracing potential root causes to their downstream consequences to discover hidden risk sources.
- Document Analysis:** Thoroughly reviewing project contracts, schedules, budgets, requirements, and architectural specifications to identify gaps or conflicting assumptions.

6 Characteristics of Software Projects

- Intangibility:** Software is not a physical product and exists primarily as logical artifacts such as requirements, designs, and code.
- High Complexity:** Software consists of numerous interconnected modules where a minor modification can trigger unforeseen side effects.
- Changing Requirements:** Requirements evolve continuously as clients refine their needs or business conditions shift.
- High Flexibility:** Software is malleable and easy to alter, which frequently encourages unmonitored scope changes.
- Difficult Progress Measurement:** Development lacks physical progress indicators and must be evaluated through completed deliverables, tested features, and defect status.
- Human-Intensive:** Heavily reliant on skilled personnel whose domain knowledge, communication, and teamwork dictate project success

Software vs. Traditional Engineering Projects

- Invisibility:** Software progress is abstract until runnable builds exist, whereas physical progress in civil or mechanical projects is directly visible.
- Conformity:** Software must adapt to legacy hardware and existing software environments rather than being governed by uniform physical laws.
- Complexity:** Software features high logical complexity per unit cost without standardized, repeated physical parts.
- Flexibility:** Software can be modified infinitely, whereas physical materials are expensive and difficult to modify once constructed.

WBS Rules & Decomposition Styles

- 100% Rule:** The WBS must encompass 100% of the work defined by the project scope—no extra work included and no required work omitted.
- Mutual Exclusion:** Each WBS element must represent distinct work to avoid double-counting tasks.
- Work Package:** The lowest WBS level where work can be estimated, scheduled, assigned, and monitored.
- Product-Oriented WBS:** Decomposes work by major software components (e.g., User Interface, Database, Server).

Software Project Scheduling

WBS Rules & Decomposition Styles

- 100% Rule:** The WBS must encompass 100% of the work defined by the project scope—no extra work included and no required work omitted8.
- Mutual Exclusion:** Each WBS element must represent distinct work to avoid double-counting tasks9.
- Work Package:** The lowest WBS level where work can be estimated, scheduled, assigned, and monitored9.
- Product-Oriented WBS:** Decomposes work by major software components (e.g., User Interface, Database, Server)9.
- Process-Oriented WBS:** Decomposes work by development phases (e.g., Requirements, Design, Coding, Testing)910.
- Hybrid WBS:** Combines product deliverables at one level and process activities at another10.

PDM / AON Dependency Types

- Finish-to-Start (FS):** Successor task cannot start until the predecessor task finishes1112.
- Start-to-Start (SS):** Successor task cannot start until the predecessor task starts11.
- Finish-to-Finish (FF):** Successor task cannot finish until the predecessor task finishes11.
- Start-to-Finish (SF):** Successor task cannot finish until the predecessor task starts11.

4 Categories of Dependencies

- Mandatory Dependency:** Hard logic inherent in the nature of the work (e.g., code must be written before it can be tested)1213.
- Discretionary Dependency:** Soft logic defined by the project team based on preference or best practices13.
- External Dependency:** Precedence relationship driven by external non-project entities (e.g., government regulations)1314.
- Internal Dependency:** Precedence relationship between internal project activities14.

Schedule Compression Techniques

- Project Crashing:** Adding extra resources to critical path activities to shorten duration at the expense of higher costs1516.
- Fast Tracking:** Executing critical tasks in parallel that were originally planned sequentially, increasing the risk of rework16.

PERT vs. CPM

- Critical Path Method (CPM):** Deterministic scheduling technique using a single time estimate to identify the longest sequence of dependent tasks with zero float17more_horiz.
- Program Evaluation and Review Technique (PERT):** Probabilistic scheduling technique using 3 time estimates (Optimistic \$a\$, Most Likely \$m\$, Pessimistic \$b\$) for projects with uncertain activity durations15more_horiz.

6 Primary Effort Estimation Techniques

- Expert Judgment:** Estimating effort based on personal background, domain experience, and historical memory.
- Estimating by Analogy:** Comparing the proposed project to similar past completed projects.
- Top-Down Estimating:** Estimating total project effort first and dividing it among major development phases.
- Bottom-Up Estimating:** Estimating individual WBS work packages separately and summing them up.
- Function Point Analysis (FPA):** Sizing software based on user functionality (Inputs, Outputs, Inquiries, Internal Files, External Interfaces) independent of programming language.
- COCOMO Model:** Algorithmic model estimating effort in Person-Months based on software size in KSLOC.

Risk Management

3 Categories of Risk

- Project Risks:** Threats affecting management, schedules, budgets, staffing, and resources.
- Product / Technical Risks:** Threats affecting software quality, architecture, security, performance, or third-party components.
- Business Risks:** External commercial threats such as competitor releases, lost funding, or changing market demands.

4 Threat Response Strategies

- Risk Avoidance:** Changing project scope or approach to eliminate the risk completely.
- Risk Mitigation:** Taking proactive steps to reduce the probability or impact severity of a risk.
- Risk Transfer:** Shifting financial impact or ownership to a third party (e.g., outsourcing, insurance).
- Risk Acceptance:** Acknowledging the risk without proactive changes while establishing contingency buffers.

software Quality Management

Quality Frameworks & Models

- Total Quality Management (TQM):** Organization-wide philosophy emphasizing customer focus, continuous process improvement, and shared quality responsibility.
- Six Sigma (DMAIC):** Data-driven quality framework targeting 3.4 Defects Per Million Opportunities using **Define, Measure, Analyze, Improve, and Control**.

ISO 9126 Software Quality Characteristics

- Functionality:** Ability to provide functions satisfying stated and implied needs (suitability, accuracy, security).
- Reliability:** Ability to maintain required performance levels without failure under specified conditions.
- Usability:** Ease with which users can understand, learn, and operate the system.
- Efficiency:** Response time and performance relative to consumed system resources.
- Maintainability:** Ease of analyzing, modifying, correcting, or enhancing code.
- Portability:** Ease of transferring software to another operating environment.

Walker Royce's 7 Core Metrics

- Work and Progress:** Tracks completed deliverables against planned milestones.
- Budgeted Cost:** Compares planned expenditures against actual spending.
- Staffing Dynamics:** Monitors headcount and team composition shifts over time.
- Change Traffic:** Measures the volume and rate of requirement or artifact modifications.
- Breakage:** Tracks defects, failures, or damage introduced during development.
- Rework:** Measures effort spent redoing or correcting completed work.
- MTBF / Maturity:** Evaluates system reliability using Mean Time Between Failures.

Software Configuration Management

The 4 Elements of SCM

- Configuration Identification:** Defining and uniquely labeling Configuration Items (CIs) and baselines.
- Configuration Change Control:** Evaluating, approving, and recording modifications formally via a Configuration Control Board (CCB).
- Configuration Status Accounting:** Maintaining logs of who changed what, when, why, and which version is active.
- Configuration Audit:** Conducting Functional (FCA) and Physical (PCA) audits to verify compliance with approved baselines.

The 6-Step SCM Change Control Workflow

- Change Request (CR):** Stakeholder formally submits a document describing a proposed modification.
- CCB Evaluation & Impact Analysis:** CCB analyzes impacts on cost, schedule, security, and dependent CIs.
- CCB Approval or Rejection:** CCB formally approves, rejects, or defers the change request.
- Check-out & Code Modification:** Developer checks out the CI, implements changes, and performs developer testing.
- Review, Testing & Audit:** Code undergoes peer review, regression testing, and verification audits.
- Check-in & Baseline Update:** Modified CI is checked back in, and the official baseline moves forward.

Version Control System Architectures

- CVS:** Centralized system with file-by-file commits (non-atomic), heavy branching, and required server connectivity.
- SVN:** Centralized system featuring atomic commits and versioned directories, but dependent on a central server for operations.
- Git:** Distributed system providing full local repository copies on every client, lightweight branching, and full offline operations.

4 Key Project Evaluation Questions

- Is it strategically valuable?:** Determines alignment with long-term business goals and competitive position.
- Is it technically feasible?:** Evaluates whether the system can be developed using available technology, infrastructure, and technical skills.
- Is it financially worthwhile?:** Verifies whether expected economic benefits outweigh development and operational costs.
- Are its risks acceptable?:** Assesses whether project uncertainties are manageable relative to returns.

Cost-Benefit Evaluation Metrics

- Payback Period:** The time required for a project to recover its initial investment from net cash inflows.
- Return on Investment (ROI):** Percentage measuring project profitability relative to total investment.
- Net Present Value (NPV):** Sum of discounted future net cash flows minus initial investment, accounting for the time value of money

Software Process Models

- Waterfall Model:** Sequential model suitable when requirements are stable and well-understood.
- Spiral / Risk-Driven Model:** Iterative model focusing heavily on risk identification and reduction in large, complex systems.
- Agile Methods:** Iterative model delivering working increments rapidly when requirements change frequently

Core Responsibilities Project manager

- Technical Competence:** Evaluates technical feasibility, architectural choices, and technical risks to make informed decisions without micromanaging specialists.
- Planning & Organizing:** Defines scope, constructs the Work Breakdown Structure (WBS), estimates effort, sets schedules, and allocates resources.
- Leadership & Team Management:** Provides direction, builds trust, handles team conflicts, and creates an environment where issues can be discussed openly.
- Communication:** Serves as the central link connecting customers, senior management, developers, testers, analysts, and vendors.
- Risk Management:** Identifies potential uncertainties early, calculates risk exposure, develops response plans, and monitors risk triggers.
- Problem-Solving & Decision-Making:** Evaluates trade-offs between cost, time, and quality, focusing on resolving underlying root causes rather than treating symptoms.
- Negotiation & Adaptability:** Negotiates realistic compromises when client feature requests conflict with deadlines, and adapts plans as requirements change.
- Ethical Responsibility:** Maintains complete honesty regarding project status, schedule delays, costs, risks, and technical limitations.

Good PM vs. Successful Software PM

- Good Project Manager:** Demonstrates standard administrative practices like planning, organizing, communicating, and controlling.
- Successful Software Project Manager:** Combines administrative management with technical understanding, leadership, adaptability, and proactive risk management to achieve objectives in a changing software environment.

BEST OF
LUCK

